

Migrating a monolithic LAMP (Linux, Apache, MySQL, PHP) stack e-commerce application from Google Compute Engine (GCE) to Google Kubernetes Engine (GKE) or Cloud Run is a textbook example of an **"Improve and Move" (replatforming)** strategy.

Because you are moving from a Virtual Machine (VM) environment to a containerized environment, the migration requires untangling stateful components from the application layer. Based on Google Cloud's official migration framework, here is a detailed research guide on how to approach this across the four phases: **Assess, Plan, Deploy, and Optimize**.

<https://cloud.google.com/blog/products/cloud-migration/guide-to-all-google-cloud-migration-guides>

Strangler Fig Pattern: The Strangler Fig pattern provides a controlled and phased approach to modernization. It allows the existing application to continue functioning during the modernization effort. A façade (proxy) intercepts requests that go to the back-end legacy system. The façade routes these requests either to the legacy application or to the new services.

This pattern reduces risks in migration by enabling your teams to move forward at a pace that suits the complexity of the project. As you migrate functionality to the new system, the legacy system becomes obsolete, and you decommission the legacy system.

1. The Strangler Fig pattern begins by introducing a façade (proxy) between the client app, the legacy system, and the new system. The façade acts as an intermediary. It allows the client app to interact with the legacy system and the new system. Initially, the façade routes most requests to the legacy system.
2. As the migration progresses, the façade incrementally shifts requests from the legacy system to the new system. With each iteration, you implement more pieces of functionality in the new system.

This incremental approach gradually reduces the legacy system's responsibilities and expands the scope of the new system. The process is iterative. It allows the team to address complexities and dependencies in manageable stages. These stages help the system remain stable and functional.

3. After you migrate all of the functionality and there are no dependencies on the legacy system, you can decommission the legacy system. The façade routes all requests exclusively to the new system.
4. You remove the façade and reconfigure the client app to communicate directly with the new system. This step marks the completion of the migration.

<https://www.netapp.com/blog/cvo-blg-cloud-migration-approach-rehost-refactor-or-replatform/>

https://docs.cloud.google.com/architecture/migration-to-gcp-getting-started?e=48754805#types_of_migrations

Phase 1: Assess

In this phase, you are evaluating the current state of your monolith to identify dependencies, risks, and blockers for containerization.

- **Cataloging the Workload:** Understand the specific versions of Linux, Apache, PHP, and MySQL running on your GCE instances. Identify the extensions and libraries your PHP application requires.
- **Stateful vs. Stateless Analysis (The Monolith Challenge):** A typical e-commerce LAMP monolith relies heavily on local VM storage. You must audit three critical components before containerizing:
 - **User Uploads/Product Images:** Does the application save product images to a local directory (e.g., `/var/www/html/images`)?
 - **Session Management:** Are user shopping cart sessions stored on the local file system (default PHP behavior)?
 - **Database:** Is MySQL running on the exact same GCE VM as the web server, or is it on a dedicated GCE VM?
- **TCO and Business Impact:** Evaluate the expected traffic patterns. E-commerce sites often experience highly variable traffic (e.g., holiday sales). Moving to GKE or Cloud Run will drastically lower compute costs during off-peak times through scale-to-zero (Cloud Run) or node autoscaling (GKE).

Phase 2: Plan

This phase involves designing the new cloud-native foundational architecture and choosing the right deployment vehicle.

- **GKE vs. Cloud Run Decision:**
 - *Cloud Run:* Best for pure HTTP-driven web applications. It abstracts away infrastructure, scales up rapidly for sudden traffic spikes (ideal for flash sales), and scales to zero to save costs.
 - *GKE:* Best if your monolith has background workers, requires complex internal networking, or runs specific cron jobs that cannot be easily adapted to Cloud Run's request-driven model.
- **Decoupling Strategy (Crucial for Containers):**
 - *Storage:* Plan to rewrite the application's file-handling logic to use **Cloud Storage** for product images instead of local storage.
 - *Sessions:* Plan to offload PHP session states to **Memorystore (Redis)** so that when users hit different containers, their shopping carts persist.
 - *Database:* Plan a move to **Cloud SQL for MySQL** as a fully managed database service to eliminate the overhead of patching and backing up database VMs.
- **Networking & Security:** Design a Virtual Private Cloud (VPC) topology. Plan to use Serverless VPC Access (if using Cloud Run) so the containers can securely communicate with Cloud SQL and Memorystore via private internal IPs.

Phase 3: Deploy

This is the execution phase where you containerize the code and migrate the data. Google recommends utilizing Infrastructure as Code (IaC) like Terraform for deployment.

- **Step 1: Containerization:**
 - Write a **Dockerfile** that packages Linux (e.g., Alpine or Debian), Apache, PHP, and the necessary extensions.
 - Exclude the database layer and externalize configuration variables (e.g., database credentials) using **Secret Manager**.
 - Push your built container images to the Artifact **Registry**.
- **Step 2: Database Migration:**
 - Use the **Database Migration Service (DMS)** to seamlessly migrate your database from the GCE VM to Cloud SQL. DMS uses native MySQL replication to keep the target Cloud SQL database synchronized with your live GCE database until you are ready to cut over.
- **Step 3: CI/CD Pipeline:**
 - Set up **Cloud Build** to automatically build a new container image and deploy it to GKE or Cloud Run whenever a developer commits code to the repository.
- **Step 4: The Cutover:**
 - Once the new environment is provisioned and tested, lock the GCE database for writing, let the final replication sync to Cloud SQL, and update your DNS (Cloud DNS) to point traffic to your new Cloud Run service or GKE Ingress load balancer.

Phase 4: Optimize

After your e-commerce platform is successfully serving live traffic on containers, you can shift your focus to day-two operations.

- **Performance Optimization:** * Place **Cloud CDN** (Content Delivery Network) in front of your Cloud Storage bucket and load balancer. This will cache static assets (CSS, JS, product images) at the edge, drastically reducing latency for shoppers and minimizing backend load on your containers.
- **Autoscaling Tuning:**
 - *Cloud Run:* Fine-tune the **max-instances** and **concurrency** settings to ensure the application handles peak traffic without exhausting database connections.
 - *GKE:* Implement Horizontal Pod Autoscaler (HPA) and Cluster Autoscaler to automatically provision new nodes when CPU/Memory utilization spikes.
- **Observability:** Implement **Google Cloud Observability** (formerly Stackdriver). Because containers are ephemeral, local logs disappear when a container shuts down. Configure your application to stream logs to Cloud Logging and set up Cloud Monitoring alerts to notify you if the 5xx error rate spikes during checkout.

